TRABAJO PRÁCTICO 06

Conjunto de pruebas automatizadas
Convención de estilos y calidad de código
Justificación de las decisiones de diseño

EcoHarmony Park

Inscribirme a actividad

Universidad Tecnológica Nacional - Facultad Regional Córdoba

Ingeniería de Software | Comisión 4K1 | Grupo 04 | 2026

Conjunto de pruebas automatizadas, Convención de estilos y calidad de código, Justificación de las decisiones de diseño

Índice

Conjunto de pruebas automatizadas 3
1. Objetivo y alcance 3
2. Organización y ejecución 3
3. Datos y resultados comunes 4
4. Pruebas del servicio de inscripción 5
5. Pruebas del generador QR 8
6. Pruebas del resumen público 9
7. Pruebas de la API 9
8. Pruebas del correo 10
9. Pruebas de navegación y participantes 10
10. Alcance de la verificación 11

Convención de estilos y calidad de código 12
1. Propósito y alcance 12
2. Entorno de trabajo y organización 12
3. Formato del código 12
4. Nombres e identificadores 13
5. Sintaxis y módulos JavaScript 14
6. Comentarios y documentación 14
7. Validación y manejo de errores 14
8. Escritura de pruebas 15
9. HTML, CSS y presentación 15
10. Identidad visual solicitada por el PO 16
11. Verificación y mantenimiento 17

Justificación de las decisiones de diseño 18
1. Propósito y contexto 18
2. Lenguaje y organización 18
3. Servicio y contrato de inscripción 19
4. Validaciones y comunicación de errores 19
5. Almacenamiento y cupos 20
6. Fechas, horarios y reloj 21
7. QR y resumen 21
8. Confirmación por correo 22
9. Interfaz e identidad visual 23
10. Precarga de inscripciones 23
11. Estrategia de verificación 24
12. Condiciones de operación 24

Conjunto de pruebas automatizadas

1. Objetivo y alcance

Este documento describe los casos implementados en la suite de la User Story «Inscribirme a actividad». Cada identificador corresponde a una prueba `it` del código. Cuando una prueba incluye varios valores representativos, estos se detallan dentro del mismo caso.

El conjunto comprende 63 pruebas distribuidas en siete archivos. Incluye validaciones públicas, reglas de inscripción, colaboración con el repositorio en memoria, solicitudes HTTP locales, generación y lectura de QR, preparación del correo y funciones de navegación.

Las pruebas del servicio usan el repositorio real en memoria y el generador QR real, junto con un reloj controlado y una bandeja de correo simulada. Por ello no se clasifican todas como unitarias puras. Las pruebas HTTP levantan servidores locales con puerto asignado por el sistema. El adaptador de correo se comprueba con un cliente sustituto de Resend.

2. Organización y ejecución

Las rutas de esta sección son relativas a `Implementacion/`.

Archivo en test/	Prefijo de casos	Cantidad
`servicioInscripcion.test.js`	S	42
`generarQr.test.js`	Q	4
`servidorResumenInscripcion.test.js`	R	4
`apiInscripciones.test.js`	A	4
`emisorCorreoResend.test.js`	C	3
`validacionAvanceFormulario.test.js`	N	5
`participantes.test.js`	P	1
Total		63

Los números siguen el orden de declaración dentro de cada archivo. El título descriptivo de cada caso permite localizar su implementación sin depender de números de línea.

Para instalar y ejecutar con Node.js 24:

```
npm ci
npm test
```

Durante el desarrollo se utiliza `npm run test:watch`. El formato y las reglas estáticas se comprueban con `npm run format:check` y `npm run lint`. La ejecución de la suite no requiere una API key ni envía correos externos.

3. Datos y resultados comunes

3.1 Preparación del servicio

Cada caso del servicio inicializa un repositorio y una bandeja independientes. El reloj base es el miércoles 14/10/2026 a las 10:00, con offset -03:00. Se habilitan turnos a las 14:00 y 15:00 de ese día y a las 14:00 del día siguiente.

Los datos base son Ana Gómez, DNI 30111222, edad 30, y Juan Pérez, DNI 28444555, edad 34. La solicitud habitual selecciona Safari el 14/10/2026 a las 14:00, contacto `ana@ejemplo.com` y términos aceptados para Safari.

Actividad	Capacidad usada	Edad mínima	Talle
Safari	8	0	No requerido
Jardinería	12	0	No requerido
Palestra	12	12	XS, S, M, L, XL o XXL
Tirolesa	10	8	XS, S, M, L, XL o XXL

Estos valores describen la configuración probada. Las reglas no pertenecen a la convención de estilos. Cada test altera los datos necesarios para demostrar su escenario.

3.2 Criterios de comprobación

En una aceptación se observan el identificador, los datos guardados, los cupos o el mensaje según el objetivo del caso. En un rechazo se comprueba la excepción y que no exista un registro ni una confirmación nuevos; los casos de cupos, duplicación y ciertos datos inválidos también verifican explícitamente el saldo restante.

Una comprobación del error esperado representa un rechazo correcto de la funcionalidad. Ese test debe terminar aprobado. No se confunde una inscripción rechazada con una prueba fallida.

Las fechas se fijan en los tests; los escenarios de límites modifican el reloj antes de llamar al servicio. Los servidores se cierran al finalizar cada prueba HTTP.

4. Pruebas del servicio de inscripción

4.1 Registro y confirmación

ID	Preparación y acción	Resultado comprobado
S01	Inscribir una solicitud válida con Ana y Juan.	Identificador presente, solicitud recuperable, seis cupos restantes y un mensaje al contacto con participantes y adjunto.
S02	Inscribir a Ana con contacto declarado.	Un mensaje dirigido a ana@ejemplo.com.
S03	Inscribir dos participantes y consultar la confirmación.	Actividad Safari, inicio de turno y lista de participantes correctos.
S04	Inscribir un grupo de cinco personas válidas.	Una sola confirmación para todo el grupo.

4.2 Nombre, DNI, edad y estructura

ID	Preparación y acción	Resultado comprobado
S05	Llamar validarNombre con «María José Pérez Gómez», vacío, espacios, null, Ana123 y Ana@.	Solo el nombre formado por letras y espacios resulta válido.
S06	Inscribir un nombre vacío individual y un nombre con símbolo en el segundo participante.	Ambas solicitudes se rechazan; sin registro ni correo.
S07	Llamar validarDni con 7500000, 1000000, 99999999, 999999, 100000000, abc, 7500000.5 y 07500000 como texto.	Se aceptan los tres primeros; se rechazan los restantes.
S11	Llamar validarEdad con 0, 120, -1, 121, 10.5, texto, cadena numérica, vacío, null, undefined, NaN e Infinity.	Solo los enteros numéricos de 0 a 120 resultan válidos.
S12	Inscribir una edad negativa individual y texto en el segundo participante de un grupo.	Rechazo identificado por participante, sin registro ni correo.
S13	Inscribir DNI corto, alfanumérico en el segundo participante y con cero inicial.	Rechazo; el cero inicial identifica campo dni y participante; cupo conservado.
S14	Inscribir lista vacía, participantes null, un participante null, solicitud null y argumento omitido.	Error controlado, sin registro ni correo.

ID	Preparación y acción	Resultado comprobado
S15	Inscribir con actividad desconocida.	Rechazo por actividad inexistente, sin registro ni correo.
S16	Solicitar Palestra con una persona de 11 años y luego otra de 12, con talles.	Rechazo por edad inferior y aceptación en el límite.
S17	Solicitar Tirolesa con 7 años y luego con 8, indicando talle.	Rechazo por edad inferior y aceptación en el límite.
S18	Inscribir edades 0 y 120 en Safari y edad 0 en Jardinería en otro turno.	Se aceptan las solicitudes y se generan dos confirmaciones.

4.3 Duplicación y simultaneidad

ID	Preparación y acción	Resultado comprobado
S08	Registrar a Ana en Safari e incluirla segunda en Jardinería al mismo inicio.	Rechazo del grupo completo; inscripción anterior intacta, sin segundo registro ni nuevo mensaje.
S09	Repetir la misma persona, actividad y turno ya registrados.	Error de horario para participante 1; siete cupos y una sola confirmación.
S10	Después del registro inicial, cambiar la persona, el horario o la fecha en solicitudes válidas.	Se admiten las tres alternativas; cuatro mensajes en total.
S39	Repetir el DNI de Ana en la tercera posición del grupo.	Error de dni para participante 3; sin registro, sin correo y ocho cupos.

4.4 Términos y vestimenta

ID	Preparación y acción	Resultado comprobado
S19	Solicitar las cuatro actividades con aceptación falsa, ausente o cadena true.	Se rechazan antes de registrar; no se envían mensajes.
S20	Inscribir Safari con términos asociados a Jardinería.	Rechazo por aceptación de otra actividad.
S21	Solicitar Safari aceptando sus términos.	Solicitud guardada y una confirmación.
S22	Omitir talle en Palestra y en el segundo participante de Tirolesa.	Rechazo e identificación del participante; sin registro ni correo.

ID	Preparación y acción	Resultado comprobado
S23	Solicitar Palestra con talle XXXL.	Rechazo por talle no admitido.
S24	Solicitar Safari y Jardinería sin indicar talle.	Ambas inscripciones se aceptan.
S25	Usar XS, S, M, L, XL y XXL en grupos de Palestra y Tirolesa en horarios distintos.	Se aceptan todos los talles y se generan dos mensajes.

4.5 Turnos y cupos

ID	Preparación y acción	Resultado comprobado
S26	Solicitar un turno inexistente y otro deshabilitado.	Ambas solicitudes se rechazan sin registro ni correo.
S27	Dejar un lugar en un turno de dos y solicitar un grupo de dos personas.	Rechazo completo; el registro previo y el lugar libre se conservan.
S28	Registrar dos personas en Safari con ocho lugares.	Saldo de seis; se preservan los cupos de otra actividad y otro horario.
S29	Solicitar dos personas en un turno con exactamente dos lugares.	Registro completo, saldo cero y una confirmación.
S30	Solicitar turnos dados de alta en lunes, 25/12 y 01/01.	Rechazo por cierre; sin nuevo registro ni correo.
S31	Solicitar inicios a las 08:00, 18:00 y 14:30.	Rechazo por horario fuera del rango o por inicio no entero.
S32	Fijar el reloj exactamente 15 minutos antes del turno.	Inscripción aceptada y confirmación generada.
S33	Fijar el reloj un milisegundo después del límite de 15 minutos.	Rechazo por ventana cerrada, con cupos intactos.
S34	Fijar el reloj exactamente 24 horas antes del turno.	Inscripción aceptada y confirmación generada.
S35	Fijar el reloj un milisegundo antes de la apertura de 24 horas.	Rechazo por ventana aún no abierta, con cupos intactos.

4.6 Contacto, QR y concurrencia

ID	Preparación y acción	Resultado comprobado
S36	Inscribir dos solicitudes con contactos distintos.	Cada confirmación usa el contacto de su solicitud.
S37	Validar correos .com y .com.ar, sin arroba, sin dominio, null, vacío, espacios, undefined y salto de línea final.	Se aceptan los formatos válidos y se rechazan las entradas restantes.
S38	Inscribir con contacto mal formado, null, vacío y omitido.	Error en contacto; sin registro, consumo de cupo ni confirmación.
S40	Registrar una inscripción individual y otra grupal, y decodificar sus adjuntos.	Un QR por inscripción; cada enlace corresponde al código público del registro y conserva los datos correctos.
S41	Iniciar dos solicitudes grupales para los últimos dos cupos antes de esperar sus resultados.	Una aceptación y un rechazo; saldo cero, un registro y una confirmación.
S42	Iniciar Safari y Jardinería para el mismo DNI y horario antes de esperar.	Una aceptación y un rechazo por simultaneidad; solo se consume el cupo de la primera.

Las pruebas concurrentes usan `Promise.allSettled` y comprueban el comportamiento del mismo proceso y repositorio en memoria. No acreditan coordinación entre varios servidores.

5. Pruebas del generador QR

ID	Preparación y acción	Resultado comprobado
Q01	Generar un pase con el contenido INSCRIPCION-1.	Firma PNG válida y dimensiones de 1520 por 1100 píxeles, leídas con pngjs.
Q02	Generar y decodificar el pase con jsQR.	Recuperación exacta del contenido original.
Q03	Generar un enlace de resumen y reducir la imagen a 390 píxeles de ancho.	El decodificador recupera el mismo enlace.
Q04	Generar con vacío, espacios y null.	Rechazo de todas las entradas.

Q03 verifica una reducción digital concreta. No demuestra por sí sola la lectura con todas las cámaras, tamaños físicos, distancias o aplicaciones de escaneo.

6. Pruebas del resumen público

ID	Preparación y acción	Resultado comprobado
R01	Guardar una inscripción y consultar su código público por HTTP.	Respuesta 200 con actividad, participantes, inicio y fin, foto y CSS; sin DNI ni contacto, con no-store y política de recursos propios. Foto y CSS responden correctamente.
R02	Inscribir mediante el servicio, decodificar el QR y solicitar su enlace.	Respuesta 200 con el participante registrado.
R03	Guardar directamente un nombre con una etiqueta HTML y consultar el resumen.	El texto se escapa y no se inserta como etiqueta ejecutable; encabezado CSP restrictivo.
R04	Consultar un código público desconocido.	Respuesta 404 con mensaje de inscripción no encontrada.

R03 prepara el dato directamente en el repositorio para verificar la salida HTML aunque el servicio rechace ese nombre en la entrada normal.

7. Pruebas de la API

ID	Preparación y acción	Resultado comprobado
A01	Enviar una solicitud válida a POST /api/inscripciones y obtener su QR.	Respuesta 201 con identificador, enlaces de resumen y QR; el PNG responde 200 y codifica la URL devuelta.
A02	Enviar un participante sin nombre.	Respuesta 422 con campo nombre, mensaje y número de participante.
A03	Registrar una persona y consultar GET /api/turnos.	El turno de Safari a las 14:00 informa siete lugares.
A04	Habilitar precarga, consultar turnos, intentar un DNI ya ocupado y registrar en el último lugar de otro turno.	Ocupaciones variadas y sin correo inicial; rechazo por conflicto, saldo que baja de uno a cero y una sola confirmación nueva.

A04 también verifica que una consulta posterior no vuelva a reponer la ocupación inicial. La precarga se habilita solo en esa preparación; los demás casos trabajan sin ella.

8. Pruebas del correo

ID	Preparación y acción	Resultado comprobado
C01	Enviar con cliente sustituto que acepta el mensaje.	Identificador devuelto, destinatario y asunto correctos, clave de idempotencia, foto JPEG de 1200 por 675, QR inline y copia adjunta; HTML escapado y datos de fecha y hora.
C02	Usar un cliente sustituto que devuelve rechazo de Resend.	ErrorEnvioCorreo con mensaje controlado y causa del proveedor.
C03	Preparar confirmación de Tirolesa con dos personas y talles.	Fotografía correspondiente, cantidad, nombres y talles en HTML/texto, sin enlaces al servidor local para las imágenes.

Las imágenes inline y la copia descargable contienen el mismo pase; no representan dos inscripciones ni dos QR de negocio. Estos tests verifican la preparación y el contrato del adaptador. La recepción real depende de la configuración y respuesta del proveedor.

9. Pruebas de navegación y participantes

ID	Preparación y acción	Resultado comprobado
N01	Solicitar avance desde fecha y horario sin selección.	Mensaje que solicita elegir horario.
N02	Solicitar avance con un participante sin DNI.	Mensaje de datos obligatorios incompletos.
N03	Completar participantes, talle requerido, contacto y aceptación.	Avance permitido, representado por null.
N04	Volver desde fecha y horario a participantes con turno elegido.	Navegación permitida.
N05	Intentar abrir confirmación desde actividad sin elegir turno.	Navegación bloqueada por horario faltante.
P01	Eliminar el segundo de tres participantes.	Permanecen el primero y el tercero en orden; la colección original no se modifica.

Estos casos invocan funciones de presentación directamente. No ejecutan un navegador ni comprueban la animación de eliminación o todos los controles del DOM.

10. Alcance de la verificación

El inventario anterior corresponde al código automatizado de esta entrega. Las alternativas agrupadas no se cuentan como pruebas independientes. Un resultado verde acredita las aserciones ejecutadas, no un porcentaje global de cobertura ni ausencia de todo defecto.

La revisión manual complementaria comprende el recorrido en celular y escritorio, fotos y videos, foco de teclado, términos, eliminación animada, mensajes de error, contraste, correo recibido y escaneo físico del pase.

El uso real de Resend, la persistencia después de reiniciar, la operación entre varios procesos y el escaneo en dispositivos físicos no están acreditados por estas 63 pruebas. El repositorio en memoria se reinicia al cerrar el servidor. La suite del emisor no realiza llamadas de red al proveedor ni certifica entrega en una bandeja real.

El comportamiento que conserva el registro si falla el envío de correo está implementado en el servicio, pero no tiene un caso específico de esa colaboración dentro de la suite actual. El rechazo del proveedor sí se prueba de manera independiente en C02.

La cronología de trabajo se consulta en el historial del repositorio. Este documento describe los escenarios y sus resultados esperados; no reconstruye fases de desarrollo a partir del estado final del código.

Convención de estilos y calidad de código

1. Propósito y alcance

Esta convención establece criterios comunes para escribir, organizar y revisar el código de la User Story «Inscribirme a actividad». Se aplica a los módulos JavaScript del servicio, las pruebas, la interfaz web y las plantillas HTML del correo.

Su finalidad es mantener una lectura uniforme y facilitar el mantenimiento entre integrantes. Las restricciones de inscripción pertenecen a los requisitos de la funcionalidad. La elección de componentes se explica en el documento de decisiones de diseño. La enumeración de escenarios se desarrolla en el conjunto de pruebas.

El Markdown constituye la fuente editable; el PDF es su versión para presentación.

2. Entorno de trabajo y organización

Se utiliza JavaScript con módulos ES sobre Node.js 24, declarado en `package.json` y `.nvmrc`. Las dependencias resueltas se registran en `package-lock.json`; la instalación reproducible se realiza con `npm ci`.

La base del proyecto es `Entregas/Trabajos_Practicos/TP_06/Entrega/`.

Directorio	Contenido
`Implementacion/src/`	Servicio y adaptadores.
`Implementacion/test/`	Pruebas automatizadas.
`Implementacion/prototipo/`	Interfaz y servidor local.
`Implementacion/prototipo/assets/`	Medios, fuentes y licencias.
`Documentacion/`	Documentos editables y versiones PDF.

Los documentos siguen `ISW_TP_06_<Tema>_G04.md` y el mismo nombre base para su PDF. No se incluyen dependencias instaladas, cachés ni credenciales en Git. El archivo local `.env` se mantiene ignorado y contiene configuración del entorno.

3. Formato del código

- Codificación UTF-8, finales de línea LF y sangría de dos espacios, sin tabulaciones.
- Prettier determina el formato: comillas simples, punto y coma, coma final en estructuras multilínea y paréntesis en parámetros de funciones flecha.

- Ancho objetivo de 100 columnas. Se admiten líneas mayores en URLs, expresiones regulares, cadenas y plantillas que no puedan dividirse sin alterar el contenido.
- Una declaración por línea y líneas en blanco entre bloques de responsabilidad distinta. No alinear declaraciones agregando espacios manuales.
- Llave de apertura en la misma línea que la declaración. Todas las estructuras de control llevan llaves, aun con una sola instrucción.
- Las plantillas de HTML, SVG y correo pueden ocupar varias líneas. Su indentación debe mostrar la jerarquía del documento y sus interpolaciones.

Las opciones se conservan en `.prettierrc.json`. Se respeta `.prettierignore` para excluir dependencias, recursos binarios y archivos generados.

4. Nombres e identificadores

Elemento	Regla	Ejemplo
Clase	PascalCase	`ServicioInscripcion`
Función o método	camelCase, acción reconocible	`validarNombre`
Variable o parámetro	camelCase, nombre significativo	`inicioTurno`
Constante de configuración o patrón	SCREAMING_SNAKE_CASE	`CONFIGURACION_PARQUE`
Módulo JavaScript	camelCase.js	`correoConfirmacion.js`
Archivo de pruebas	camelCase.test.js	`servicioInscripcion.test.js`
Clase CSS	kebab-case	`participant-card`
Atributo personalizado HTML	data-kebab-case	`data-action`

Los conceptos del dominio usan español, sin tildes en identificadores. Los textos visibles respetan la ortografía. Se mantienen nombres propios de bibliotecas, APIs, eventos y términos técnicos comunes.

Las propiedades de contratos existentes, los campos del estado visual y los selectores conservan nombres consistentes con sus consumidores. Todo cambio de contrato actualiza sus usos.

Se admiten nombres técnicos fijos como `package.json`, `package-lock.json`, `README.md`, `.gitignore`, `.nvmrc`, `.env`, `eslint.config.js`, `.prettierrc.json`, `.prettierignore`, `index.html`, `styles.css` y `resumen.css`.

5. Sintaxis y módulos JavaScript

- Usar `const` por defecto y `let` cuando exista reasignación. No usar `var`.
- Usar `===` y `!==`. Se admite `valor == null` para tratar `null` y `undefined` conjuntamente de forma intencional.
- Usar literales de objetos y arreglos; evitar `new Number`, `new String` y `new Boolean`.
- Declarar con nombre las funciones de módulo. Usar funciones flecha en callbacks y funciones locales; los métodos de objetos usan sintaxis abreviada.
- Preferir retornos tempranos y condiciones directas; evitar ternarios anidados.
- En recorridos imperativos, preferir `for...of`. Usar `map`, `filter` o `some` cuando expresen directamente una transformación o consulta.
- Usar `import`/`export` con exportaciones nombradas en módulos de aplicación. `export default` se admite en archivos de configuración que siguen ese formato.
- Incluir `.js` en importaciones relativas y `node:` en módulos de Node.js.
- Usar plantillas para interpolaciones y evitar concatenaciones difíciles de seguir.
- No usar `eval`, `with` ni modificar prototipos nativos.
- Expresar la asincronía con `async`/`await`. Toda promesa se espera, se retorna o tiene tratamiento explícito en el límite del evento que la inicia.

Los módulos deben tener una responsabilidad reconocible. La extracción de helpers se justifica cuando aporta significado, evita repetición o facilita una modificación.

6. Comentarios y documentación

Las clases y funciones exportadas y los métodos públicos relevantes se documentan con JSDoc. Se describen propósito, parámetros, resultado y errores con `@param`, `@returns` y `@throws` cuando correspondan. Las estructuras reutilizadas pueden documentarse con `@typedef`.

Los comentarios internos explican decisiones, restricciones de bibliotecas o consecuencias que no se deducen directamente de la instrucción. No se exige comentar helpers triviales ni repetir lo que ya expresa el nombre.

Los comentarios se mantienen junto con el código que explican. Las anotaciones de tipos ayudan a la lectura y no sustituyen las comprobaciones en ejecución.

7. Validación y manejo de errores

Las comprobaciones de entrada expresan presencia, tipo y formato con claridad. Se distingue entre propiedad omitida, `null`, texto vacío, espacios, cero y `false`. Las conversiones admitidas por un adaptador se escriben explícitamente.

Se usa `Number.isInteger` para enteros y `??` para predeterminados por ausencia. Las comprobaciones booleanas se reservan para condiciones de significado booleano.

Los fallos se comunican mediante instancias de `Error`. Si un error debe presentarse en un campo, se utilizan propiedades identificables como `name`, `campo` y `participante`. No se lanzan cadenas ni objetos sin tipo de error.

Cada `catch` tiene una respuesta concreta: traducir el fallo, conservar su causa, informar un resultado o aplicar una recuperación deliberada. El visitante recibe mensajes comprensibles sin trazas internas ni credenciales.

Las cadenas externas se escapan al insertarlas en HTML. Las claves de servicios se leen desde el entorno y no se escriben en código ni en archivos versionados.

8. Escritura de pruebas

Las pruebas usan Vitest con `describe`, `it` y `expect`. Los títulos se redactan en español y describen resultado y condición; se prefiere «debe ... cuando ...».

Se sigue la estructura preparar, ejecutar y comprobar (AAA). Los comentarios de las fases se agregan cuando ayudan a leer el caso; una validación breve puede prescindir de ellos.

Las alternativas representativas de una validación simple se agrupan con `expect` explícitos. Los límites y causas diferentes se distinguen sin multiplicar variantes equivalentes ni ocultar el escenario detrás de parametrizaciones complejas.

Los datos se definen cerca del caso. Se admiten fixtures pequeños para construir datos comunes; el dato decisivo y la acción quedan visibles en la prueba.

Se comprueban interfaces públicas, resultados y efectos observables. Los colaboradores y datos se aíslan mediante inicialización local o `beforeEach`. Los servidores temporales se cierran con `afterEach`.

Las aserciones expresan el resultado esperado. No basta comprobar ausencia de error cuando el escenario exige revisar datos o efectos. El detalle corresponde al conjunto de pruebas.

9. HTML, CSS y presentación

- Usar HTML semántico, declarar idioma y asociar etiquetas con controles.
- Usar clases CSS para presentación y atributos `data-*` para acciones y datos del comportamiento.
- Mantener estilos de interfaz en CSS. Se admiten estilos inline para valores visuales calculados, gráficos SVG y plantillas de correo que requieren compatibilidad entre lectores.
- Reutilizar colores con variables CSS. En el correo se admiten equivalentes literales porque los lectores no soportan uniformemente las variables CSS.
- Priorizar lectura y uso en celular; adaptar columnas, controles y márgenes al ancho disponible.
- Mantener foco visible, operación por teclado y mensajes que no dependan solo del color.

- Las imágenes informativas tienen texto alternativo; los iconos decorativos se ocultan de la lectura asistida.
- Respetar movimiento reducido y evitar que las animaciones impidan operar o leer un campo.
- La inspección visual complementa los controles automáticos.

10. Identidad visual solicitada por el PO

La interfaz, el resumen, el correo y el pase QR deben conservar la identidad indicada por el Product Owner.

Uso	Color	Valor
Texto principal, acciones e identidad	Verde oscuro	`#386641`
Acentos y decoración	Verde medio	`#6a994e`
Resaltados y vegetación	Verde claro	`#a7c957`
Superficies y texto sobre verde oscuro	Crema	`#f2e8cf`
Señales de marca y referencia para errores	Rojo	`#bc4749`

La paleta rige la identidad general, las pantallas del recorrido, el resumen, el correo y los gráficos propios. Se permiten transparencias y degradados derivados de ella para bordes, sombras, superposiciones y decoración. `transparent`, `inherit` y `currentColor` conservan su uso técnico.

Se admite una excepción para colores semánticos de estado: alertas, errores de validación, advertencias, confirmaciones y avisos del sistema pueden usar colores adicionales cuando ayuden a reconocer su significado. Por ejemplo, un error puede combinar texto rojo oscuro, un borde rojo y un fondo claro rojizo. Esta excepción se aplica también a una alerta dentro del formulario; su función de comunicar un estado determina el uso, no su ubicación.

Los colores adicionales se definen como variables con nombres de función, como `--error-text`, `--error-border` y `--error-background`, y se reutilizan de forma consistente. No se extienden a fondos generales, botones de navegación o decoración sin relación con el estado comunicado. Se prioriza distinguir el aviso sin competir con toda la pantalla.

Las fotografías y videos conservan sus colores originales. El gráfico QR utiliza verde oscuro y crema.

La tipografía es Lato, servida localmente en las páginas. Se declaran alternativas del sistema; el correo admite Arial o sans-serif si su lector no dispone de Lato.

El contraste mínimo es 4,5:1 para texto normal y 3:1 para texto grande, incluidos los avisos. Se usa verde oscuro sobre crema o crema sobre verde oscuro en los textos generales. Verde medio, verde claro y el rojo de marca no se usan como texto pequeño sobre crema si no alcanzan el contraste requerido. En las alertas se admite un tono específico más oscuro para asegurar la lectura. El mensaje

debe explicar el estado y, cuando corresponda, cómo resolverlo; nunca se comunica el resultado únicamente mediante color.

11. Verificación y mantenimiento

Desde `Implementacion/` se ejecutan:

```
npm run format:check
npm run lint
npm test
```

`npm run format` aplica el formato y `npm run test:watch` permite pruebas interactivas. Los cambios visuales requieren revisar lectura, disposición móvil, foco y paleta en las vistas afectadas.

Prettier y ESLint controlan las reglas configurables automáticamente. La claridad de nombres, comentarios, pruebas y presentación se revisa también entre integrantes. Aprobar las herramientas no acredita por sí solo el cumplimiento completo de la convención.

Cada actualización identifica versión y fecha y regenera su PDF, manteniendo una única fuente editable.

Justificación de las decisiones de diseño

1. Propósito y contexto

Se implementa la inscripción individual o grupal a una actividad de EcoHarmony Park. El recorrido comprende elegir actividad y turno, completar participantes y contacto, aceptar los términos y confirmar. Se genera un pase QR y se prepara la confirmación por correo.

Este documento explica las decisiones de estructura y colaboración presentes en el código, su justificación y sus consecuencias. La convención de estilos especifica cómo se escribe el código; el conjunto de pruebas enumera los escenarios automatizados.

2. Lenguaje y organización

D01. JavaScript con módulos ES

Se utiliza JavaScript tanto en el servidor Node.js 24 como en el navegador. Esta elección permite trabajar con un mismo lenguaje y mantener pruebas legibles con Vitest. Los módulos ES hacen explícitas las dependencias y permiten importar las funciones desde las pruebas.

Se conservan `package.json` y `package-lock.json` para declarar el entorno y resolver dependencias mediante `npm ci`. ESLint y Prettier automatizan las comprobaciones configurables.

D02. Separación de responsabilidades

Componente	Responsabilidad
ServicioInscripcion	Validar la solicitud y coordinar registro, QR y confirmación.
RepositorioParqueEnMemoria	Conservar turnos e inscripciones, consultar cupos y conflictos.
relojSistema	Proveer el instante actual al servicio y a la API.
generarQr	Codificar contenido y producir la imagen del pase.
EmisorCorreoSimulado / EmisorCorreoResend	Registrar mensajes de prueba o enviarlos mediante el proveedor.
crearContenidoCorreo	Construir HTML y texto del mensaje.
crearApiInscripciones	Adaptar solicitudes HTTP al caso de uso y responder resultados.
servidorResumenInscripcion	Mostrar la inscripción identificada por el enlace QR.
Interfaz en prototipo/	Mantener el estado del recorrido y presentar datos y errores.

La configuración pública del parque se comparte desde `prototipo/configuracionParque.js`. Su ubicación permite que el servidor local la entregue como módulo del navegador sin proceso de compilación. El servicio también la importa. Se congelan sus objetos para conservar los valores durante una ejecución.

Se conserva una estructura pequeña de módulos; no se agregan capas o entidades que únicamente encapsulen propiedades sin comportamiento.

3. Servicio y contrato de inscripción

D03. Objetos como datos y servicio como coordinador

La solicitud se representa con un objeto que contiene `actividad`, `inicioTurno`, `participantes`, `contacto` y `terminos`. Cada participante declara nombre, DNI, edad y talle cuando corresponde.

No se crea una clase de solicitud con validaciones duplicadas. `ServicioInscripcion` reúne el comportamiento del caso de uso y expone los métodos `validarNombre`, `validarDni`, `validarEdad` y `validarContacto`. Esto permite comprobar alternativas de cada campo directamente y verificar después su aplicación durante la inscripción.

La cantidad del grupo se obtiene de la lista de participantes; no se incorpora un contador de negocio independiente que pueda contradecir esa lista.

D04. Dependencias reemplazables

El servicio recibe repositorio y emisor, junto con opciones de reloj, generador QR y construcción de la URL del resumen. En ejecución se usan los adaptadores reales o locales; en las pruebas se controlan el tiempo y el transporte de correo.

La inyección evita que probar una regla dependa de la hora del equipo o de una llamada a un proveedor externo. No se exige una jerarquía de clases para colaboradores que ya cumplen el contrato necesario.

4. Validaciones y comunicación de errores

D05. Comprobación en el servicio

La interfaz facilita completar los campos y bloquea el avance si faltan datos obligatorios. El servicio comprueba definitivamente la estructura, los formatos, la actividad y las condiciones del turno antes de registrar.

Esta separación permite recibir solicitudes desde el formulario o desde otro cliente HTTP con el mismo resultado. La validación del navegador no constituye garantía suficiente.

D06. Tipos y formatos explícitos

El DNI se conserva como texto; se aceptan dígitos sin puntos ni espacios, dentro del intervalo 1000000 a 99999999 y sin cero inicial. La edad debe ser un número entero entre 0 y 120. La interfaz convierte explícitamente la edad ingresada al construir la solicitud.

El nombre acepta letras Unicode y espacios, incluido un nombre compuesto con tildes. Se rechazan ausencia, vacío, espacios solos, números y símbolos. Los espacios intermedios no se normalizan ni se consideran una restricción adicional.

El contacto se comprueba con una expresión regular de formato básico. Acepta dominios como .com y .com.ar; no verifica existencia de la casilla ni implementa todas las variantes permitidas por los estándares de correo.

Estos criterios describen los contratos implementados y probados. Los límites de entrada y las elecciones técnicas no se presentan como parte de una regla de estilo.

D07. Errores identificables y rechazo grupal

Los errores de inscripción usan `Error` con nombre `ErrorValidacion`, campo afectado y, cuando corresponde, posición del participante. El adaptador HTTP los transforma en respuesta 422 con mensaje apto para presentar en el formulario.

Se informa el primer problema encontrado. Ante una validación fallida no se registra una parte del grupo. El resultado conserva la coherencia entre participantes, ocupación y correo.

5. Almacenamiento y cupos

D08. Repositorio en memoria

Se utiliza un repositorio con mapas de turnos, inscripciones y códigos de resumen. Al guardar y recuperar datos se emplea `structuredClone` para que la interfaz o un test no modifiquen indirectamente las referencias internas.

Esta solución permite ejecutar y demostrar el caso de uso sin instalar una base de datos. Los datos permanecen durante la vida del proceso; el reinicio elimina las inscripciones y los códigos que no vuelvan a generarse.

D09. Cupos derivados de los registros

Los lugares disponibles se calculan como capacidad del turno menos participantes de las inscripciones registradas para esa actividad e inicio. No se mantiene un contador paralelo de ocupación que deba sincronizarse con cada alta.

La interfaz consulta los cupos al solicitar los horarios. El servicio vuelve a comprobarlos al confirmar. La visualización es informativa: seleccionar un turno no reserva lugares.

D10. Exclusión dentro del proceso

La comprobación de conflictos y cupos y el guardado se ejecutan sin un `await` intermedio. La generación del QR y el correo ocurren después del registro. Con el repositorio síncrono en memoria, dos llamadas concurrentes del mismo proceso no pueden aceptar ambas los últimos lugares antes de ver el registro anterior.

La simultaneidad se identifica por DNI e instante de inicio, tanto en la misma actividad como en otra. Además, se rechazan DNI repetidos dentro de la solicitud.

Esta garantía depende del proceso único y del adaptador actual. No se extiende automáticamente a varios procesos ni a una futura base de datos, que requeriría su propio control transaccional.

6. Fechas, horarios y reloj

D11. Tiempo controlable y zona del parque

La API y el servicio consultan un reloj reemplazable. El adaptador normal devuelve la hora real; los tests fijan instantes para comprobar fronteras exactas sin esperar.

Se usa la zona `America/Argentina/Buenos_Aires` en la presentación. Los turnos generados por la API se representan mediante fecha ISO con offset -03:00. El frontend consulta la hora del dispositivo para presentar opciones; el servicio usa el reloj del servidor al confirmar.

Cambiar la zona de presentación no sincroniza el reloj del dispositivo. Por eso la aceptación definitiva depende del servidor.

D12. Selección conjunta del turno

La interfaz muestra Hoy y Mañana en lugar de un calendario mensual. Cada tarjeta representa fecha y hora juntas, lo que reduce pasos y evita seleccionar primero una fecha sin opciones.

Se consideran inicios enteros entre 09:00 y 17:00, duración de una hora y ventana desde 24 horas hasta 15 minutos antes, con ambos extremos incluidos. Se excluyen lunes, 25 de diciembre y 1 de enero. El servicio comprueba estas condiciones incluso si el turno había sido seleccionado previamente.

7. QR y resumen

D13. Un pase por inscripción con enlace al resumen

Se genera un QR por inscripción, sea individual o grupal. El contenido es una URL que muestra actividad, fecha, horario y nombres del grupo, cumpliendo la respuesta del PO de visualizar un resumen al escanear.

Se utiliza un código público UUID separado del identificador secuencial interno. Esto evita exponer una secuencia de registros en los enlaces. Quien posee el enlace puede consultar el resumen; no equivale a autenticación.

El resumen omite DNI y correo de contacto. Las cadenas se escapan y se aplican encabezados para restringir recursos, evitar caché y evitar referencias a terceros desde esa vista.

D14. Composición del pase con silueta de elefante

La matriz QR se crea con `qrcode` y se integra a una silueta vectorial estable. `sharp` produce un PNG de 1520 por 1100 píxeles con verde oscuro y crema. Se conservan intactos los módulos y patrones de localización de la matriz.

El pase usa corrección de errores H y un margen configurado de un módulo, elegido para aproximar visualmente la matriz al resto de la figura. Este margen es menor que la zona libre habitual de cuatro módulos: la composición prioriza el resultado visual acordado y necesita comprobarse con los lectores físicos utilizados.

Referencia técnica: DENSO WAVE, «Point for determining the code area», https://www.qrcode.com/en/howto/code.html.

La suite decodifica el PNG con una biblioteca independiente y también una reducción a 390 píxeles de ancho. Esto verifica esos casos digitales, no todos los dispositivos. En celular el pase ocupa una fila propia; en el correo se incluye con mayor resolución.

8. Confirmación por correo

D15. Adaptador de transporte y contenido separado

`EmisorCorreoResend` implementa el envío mediante Resend. Sin clave configurada, el servidor local selecciona `EmisorCorreoSimulado`. La plantilla de contenido se mantiene en un módulo separado para modificar su presentación sin cambiar las validaciones.

La clave se lee desde `RESEND_API_KEY`. El comando del prototipo carga un `.env` local si existe; Git lo ignora. El remitente puede configurarse con `RESEND_FROM`. La clave no se entrega al navegador.

D16. Correo legible e imágenes incorporadas

Se usa HTML con tablas de presentación, estilos inline y alternativa de texto. El correo conserva la paleta y la tipografía declarada con fuentes alternativas.

La fotografía de la actividad y el QR se incorporan mediante Content-ID, evitando que el lector de correo deba consultar fotografías del servidor local. La misma imagen QR se incluye como archivo PNG descargable. Es un único pase presentado de dos maneras.

La clave de idempotencia del envío deriva de un hash del QR para identificar ese pase ante el proveedor. No se implementa un sistema de reintentos o cola de entrega.

D17. Estado del registro frente al correo

El registro se confirma antes de contactar al proveedor. Si falla el envío, el servicio conserva la inscripción y devuelve `correoEnviado: false`; la interfaz informa esa situación. Se evita eliminar una reserva válida por un problema posterior de transporte.

Si la generación del QR falla después del guardado, el registro también permanece; no hay una transacción de compensación. El emisor simulado conserva mensajes y su éxito no acredita recepción externa. La disponibilidad del proveedor y sus restricciones de remitente y destinatario son condiciones del envío real.

9. Interfaz e identidad visual

D18. Recorrido por etapas y prioridad móvil

La interfaz usa HTML, CSS y JavaScript sin framework. Se conservan cuatro etapas: Actividad, Fecha y horario, Participantes y Confirmación. El estado de los campos permanece durante la navegación para poder revisar y corregir antes de confirmar.

Los participantes adicionales se pueden eliminar individualmente, conservando el orden de los restantes. La animación breve acompaña la eliminación; la preferencia de movimiento reducido permite efectuarla sin transición.

Se prioriza el uso en celular con tarjetas y campos apilados, acciones accesibles y el QR en un bloque amplio. La disposición de escritorio aprovecha columnas y resumen lateral.

D19. Identidad indicada por el PO

Se utiliza Lato y los cinco colores especificados: verde oscuro #386641, verde medio #6a994e, verde claro #a7c957, crema #f2e8cf y rojo #bc4749. La convención exige esta paleta en la identidad general de la interfaz, el resumen, el correo y los gráficos propios.

Verde oscuro y crema forman la combinación principal de lectura. Los otros colores identifican acentos, decoración y errores. Las fotografías y videos conservan su color natural. Las transparencias se derivan de la misma paleta.

Se distinguen los colores de marca de los colores semánticos de estado. Los errores, advertencias, confirmaciones y avisos pueden usar tonos adicionales para facilitar su reconocimiento, manteniendo contraste y texto explicativo. La excepción también abarca los errores dentro del recorrido y se limita al componente que comunica el estado. Los colores de error se centralizan en variables CSS para aplicar el mismo criterio en todos los avisos; no reemplazan la identidad general del parque.

Los videos son decorativos y mantienen un respaldo fotográfico. La reproducción no condiciona la selección ni la confirmación. Las imágenes locales y sus créditos forman parte de la entrega.

10. Precarga de inscripciones

D20. Datos de presentación independientes del correo

El servidor local habilita una precarga opcional. Al crear un turno válido se agregan personas ficticias según una secuencia determinista de actividad y hora: libre, dos lugares ocupados, un lugar disponible o completo.

Las consultas posteriores no repiten esa carga ni reponen cupos consumidos. Los registros precargados afectan a cupos y conflictos igual que los registros del repositorio; no generan correos iniciales.

El resto de la API puede crearse sin precarga, como ocurre en la mayoría de tests. El reinicio reconstruye los datos para los turnos generados según la hora vigente. La secuencia no garantiza disponibilidad para toda actividad si solo queda un turno válido y a ese turno le corresponde ocupación completa.

11. Estrategia de verificación

D21. Comprobaciones públicas y alternativas representativas

Las validaciones simples se prueban agrupando valores válidos, causas de rechazo y límites. Las pruebas del servicio agregan el efecto completo de inscripción para comprobar que una validación no quede aislada de su uso real.

Las pruebas de concurrencia inician solicitudes antes de esperarlas, sin pausas artificiales. Las pruebas HTTP usan puertos temporales; el transporte externo de correo se sustituye. El QR se valida con un decodificador distinto del generador.

El conjunto de pruebas documenta 63 casos y sus límites. Las herramientas no sustituyen la inspección de la interfaz ni la comprobación del correo y del escaneo en dispositivos reales.

12. Condiciones de operación

Esta entrega funciona con un único servidor y almacenamiento en memoria. Las validaciones de cupos y conflictos observan los registros de esa ejecución. No se garantiza conservación de enlaces después de reiniciar.

El resumen requiere que la URL incluida en el QR sea accesible desde el dispositivo lector. `127.0.0.1` sirve para la misma computadora; en otro equipo apunta a ese otro equipo. La configuración `URL_PUBLICA` debe corresponder al servidor accesible para una demostración desde teléfonos u otras computadoras.

El envío externo requiere una clave y un remitente admitidos por Resend. El transporte simulado permite ejecutar el recorrido sin credenciales. El acceso público, la persistencia duradera y la coordinación distribuida constituyen capacidades distintas de las implementadas en este servidor local.